

📦 (الوراثة) Inheritance في C# أولاً:

💡 الفكرة ببساطة

(Derived Classes) وعندك أنواع منه (Base Class) "تخيل عندك" قالب عام

ممثل حياتي:

- حساب بنكي عام = BankAccount
- حساب توفير = SavingsAccount
- حساب جاري = CurrentAccount

بدل ما تكتب نفس الخصائص في كل حساب، تخليهم "يرثون" من الحساب العام

🤖 كيف نفهمها برمجياً؟

🔴 Base Class (الأب)

```
class BankAccount
{
    public double balance;

    public void Deposit(double amount)
    {
        balance += amount;
    }
}
```

👉 "هذا" القالب الأساسي

🌀 Derived Class (الابن)

```
class SavingsAccount : BankAccount
{
}
```

💡 هنا المعنى :

SavingsAccount بدون ما أكتب شيء BankAccount ورث كل شيء من

يعني:

- balance عنده
- Deposit() عنده

🧠 أهم فكرة:

”الوراثة = “إعادة استخدام الكود بدل التكرار

📦 C# أنواع الوراثة في

1) Single Inheritance

```
class Dog : Animal
```

كلب يرث من حيوان

2) Multilevel

Animal → Dog → Bulldog

يعني سلسلة

- Animal (أب)
- Dog (ابن)
- Bulldog (ابن الابن)

3) Hierarchical

Dog : Animal

Cat : Animal

أكثر من كلاس يرثون من نفس الأب

✗ Multiple Inheritance لا تدعم C#

يعني:

class C : A, B ✗

ليش؟

(ambiguity) عشان ما يصير تعارض

■ Access Modifiers (مهم جدًا في الوظائف)

● public

👉 الكل يقدر يستخدمه

● protected

👉 الأبناء فقط يقدرن يستخدمونه

● private

👉 داخل نفس الكلاس فقط

مثال سريع:

```
class Animal
{
    private int age;          // لا يُورث فعليًا
    protected string name;   // يُورث لكن للأبناء فقط
    public void Speak() {}
}
```

■ base keyword (مهم جدًا في المقابلات)

💡 فكرته:

“أرجع للأب”

استخدامه:

1. للأنشئة constructor
2. أو إنشاء دالة من الأنشئة

مثال بسيط:

```
class BankAccount
{
    public BankAccount(string name)
    {
        Console.WriteLine(name);
    }
}

class SavingsAccount : BankAccount
{
    public SavingsAccount(string name) : base(name)
    {
    }
}
```

هنا: ➡

- `base(name)` = إرسال الاسم للأنشئة أولاً

▣ Override vs New (مهم جدًا)

🕒 override = تعديل سلوك الأب (صح رسمي)

```
public virtual void Speak()
```

في الابن:

```
public override void Speak()
```

👉 "هذا" تغيير رسمي

🕒 new = إخفاء (مو تعديل)

```
public new void Speak()
```

👉 هذا يعتبر:

"أنا بس مخفي الأب وأكتب شيء جديد"

▣ Sealed (قفل الوراثة)

💡 المعنى:

"قف، ممنوع أحد يرث مني"

```
sealed class Cat
{
}
```

❌ ما تقدر تسوي

```
class Lion : Cat // خطأ
```

Abstract Class (مهم جدًا في الوظائف)

💡 الفكرة:

“كلاس ناقص، لازم تكمله أنت”

```
abstract class Animal
{
    public abstract void Speak();
}
```

👉 ما فيه تنفيذ

في الابن:

```
class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Bark");
    }
}
```

📌 الخلاصة:

النوع	المعنى
abstract	قاعدة لازم تُكْمَل
override	أُكْمَل أو غَيَّر
virtual	قابل للتغيير
base	ارجع للأب
sealed	اقفل الوراثة

📌 (تعدد الأشكال) Polymorphism ثانيًا:

💡 الفكرة ببساطة:

نفس الاسم... لكن سلوك مختلف

مثال حياتي:

“Speak”

- كلب → ينبج

- قَط → يَمُوء
- إِنْسَان → يَتَكَلَّم

نوعين:

1) Compile Time (قبل التشغيل)

Method Overloading

نفس الاسم لكن اختلاف في المدخلات

`Deposit(double amount)`

`Deposit(double amount, string checkNumber)`

👉 الجهاز يختار المناسب تلقائيًا

فكرة بسيطة:

نفس الاسم = لكن توقيع مختلف

2) Runtime Polymorphism (وقت التشغيل)

Method Overriding

```
class Animal
```

```
{
```

```
    public virtual void Speak()
```

```
}
```

```
class Dog : Animal
{
    public override void Speak()
}
```

💡 الفكرة المهمة جدًا:

```
Animal a = new Dog();
a.Speak();
```

👉 النتيجة:

“Dog Speak”

🤔 ليش؟

لأن القرار يتم وقت التشغيل

▣ Operator Overloading

💡 الفكرة:

تخلي + أو - يشتغل على كلاس

```
account1 + account2
```

👉 يعني تجمع الرصيد

☐ (الخلاصة المهمة جدًا) امتحان + مقابلات

🌀 Inheritance

- إعادة استخدام الكود
- علاقة "is-a"

🌀 Polymorphism

- نفس الوظيفة بس سلوك مختلف

📝 جملة تحفظها للمقابلة

Inheritance is for code reuse, while polymorphism is for flexible behavior at runtime.

📝 الفكرة الكبرى قبل التفاصيل

.أي نظام (مستشفى، متجر، تطبيق) = علاقات بين كلاسات

:السؤال الأساسي دائماً

كيف هذا الكلاس مرتبط بالثاني؟

:والعلاقات 4 أنواع فقط

1. ● Is-A (Inheritance)
2. ● Has-A Strong (Composition)

3.  Has-A Weak (Aggregation)

4.  Uses (Association)

 1) Is-A → Inheritance (وراثه)

 معناها:

هذا الكلاس "نوع من" كلاس آخر

مثال:

- Dog is Animal
- SavingsAccount is BankAccount

 متى استخدمه؟

"إذا الشيء" هو نفسه نوع من شيء عام

 مثال:

```
class Animal
{
    public void Eat() {}
}
```

```
class Dog : Animal
{
```

```
public void Bark() {}  
}
```

⚠️ (مقابلات):

❌ لا تستخدمه بين كائنات مشروعاتك كثير

✅ : استخدمه غالبًا مع

- Controllers
- DbContext
- Framework classes

🧠 قاعدة ذهبية:

Inheritance = is-a relationship

🔗 2) Has-A Strong → Composition

💡 معناها:

كلاس "يمتلك" كلاس ثاني بالكامل

والثاني:

❌ ما يعيش لحاله

مثال حياتي 🧑🏻:

Order يحتوي OrderItems

العناصر تختفي → Order إذا حذفنا

مثال 🖥️:

```
class Order
{
    public List<OrderItem> Items = new List<OrderItem>();
}
```

الفكرة 🧠:

- Order "هو" المالك
- Items جزء منه فقط

متى أستخدمه؟ ⚠️

إذا:

الكائن لا معنى له بدون الأب

قاعدة ذهبية 🏆:

Composition = strong ownership

3) Has-A Weak → Aggregation

معناها:

كلاس "يرتبط" بكلاس آخر لكن لا يملكه

مثال حياتي:

- Doctor عنده Patients
لكن المرضى يعيشون بدون الطبيب

مثال:

```
class Appointment
{
    public int doctorId;
    public int patientId;
}
```

الفكرة:

- "العلاقة مجرد" ربط
- بدون ملكية

قاعدة ذهبية:

Aggregation = reference only

○ 4) Association → Uses

💡 معناها:

كلاس “يستخدم” كلاس ثاني مؤقتًا

🔄 مثال:

فقط وقت التنفيذ Database يستخدم Service

💻 مثال:

```
class ReportService
{
    public void Generate(HospitalContext context)
    {
        Console.WriteLine(context.Patients.Count);
    }
}
```

💡 الفكرة:

- لا تخزين
- لا ملكية
- فقط استخدام مؤقت

(مقارنة سهلة جدًا (احفظها 📌)

العلاقة	المعنى	القوة
Inheritance	is-a	قوي
Composition	owns	قوي جدًا
Aggregation	has	متوسط
Association	uses	ضعيف

(أهم سؤال في المقابلات (احفظه حرفيًا 📌):

If I delete the parent, does the child still make sense?

إذا:

- ❌ لا → Composition
- ✔️ نعم → Aggregation

(مثال سريع جدًا (يميزك في المقابلة 📌)

📖 Hospital System:

Doctor ↔ Patient

- Doctor لا يملك Patient
👉 Aggregation

Order ↔ OrderItems

- Order يملك Items
👉 Composition

Controller ↔ Service

- 👉 Association (uses)

Dog ↔ Animal

- 👉 Inheritance

⚠️ (نقطة ذهبية من المادة) مهم جدًا

في المشاريع الحقيقية

- ❌ Inheritance لا تكثر بين موديلاتك
- ✅ Has-A = أغلب العلاقات

ليه؟ 🤔

لأن:

- Inheritance = ربط قوي (hard coupling)
- Has-A = مرن وأسهل للتغيير

🔹 خلاصة ذكية للمقابلة:

Use inheritance only for "is-a" relationships and framework extension.

Use composition and aggregation for business models because they keep the system flexible and loosely coupled.

ويجي كثير في المقابلات، Database والـ OOP تمام — هذا موضوع مهم جدًا لأنه هو “الجسر” بين

Backend بشرح لك بطريقة تدريب عملي كأنك داخل شركة.

أول فكرة لازم تثبتها 🧠

مباشر SQL بدون Database بـ C# طريقة تربط = EF Core

بدل ما تكتب:

```
SELECT * FROM Users
```

تكتب:

```
context.Users.ToList();
```

📦 (ORM) الفكرة الأساسية 1

💡 معناها ORM:

C# تحويل الجداول ↔ كائنات

يعني:

	Database	C#
Table		Class
Row		Object
Column		Property

مثال بسيط 🧠

```
class Product
{
    public int Id { get; set; }
}
```

```
public string Name { get; set; }  
}
```

👉 DB في Table هذا يمثل

EF Core أهم 3 مفاهيم في 2)

DbContext

“مدير الاتصال بالبيانات”

```
class AppDbContext : DbContext  
{  
    public DbSet<Product> Products { get; set; }  
}
```

DbSet

Table يمثل

DbSet<Product> Products

يعني:

👉 في البيانات Products جدول

Entity

كلاس يمثل جدول

```
class Product
```

الفكرة المهمة جدًا (3)

💡 SQL أنت لا تتعامل مع

أنت تتعامل مع

SQL يحولها إلى EF Core Objects →

(الديكوريشن) 4) Data Annotations

💡 معناها:

قوانين تصنيفها على الكلاس عشان تتحكم في قاعدة البيانات

(أهم الأنواع) احفظها كتصنيف

⦿ [Key]

[Key]

```
public int Id { get; set; }
```

👉 Primary Key

🌀 [Required]

[Required]

```
public string Name { get; set; }
```

👉 لا يسمح بـ null

🌀 [MaxLength]

[MaxLength(100)]

👉 أقصى طول للنص

🌀 [Column]

[Column("ProductName")]

👉 DB يغير اسم العمود في

🌀 [Table]

[Table("Products")]

👉 يغير اسم الجدول

🌀 [ForeignKey]

```
public int CategoryId { get; set; }
```

👉 يربط جدول بجدول ثاني

🌀 [NotMapped]

[NotMapped]

```
public string Temp { get; set; }
```

👉 لا يدخل الداتابيس

🌀 [Index]

[Index(IsUnique = true)]

👉 يمنع التكرار

🌀 [Range]

[Range(1,100)]

👉 يحدد مدى القيم

📦 5) Migrations (مهم جدًا في الوظيفة)

💡 معناها:

Database تحويل الكود إلى

:الأوامر

1) إنشاء Migration

add-migration Init

2) تطبيق على DB

update-database

3) حذف آخر Migration

remove-migration

الفكرة:

Migration = كل تغيير في الكود

6) EF Core العلاقات في

DB العلاقات لكن في OOP هنا يدخل نفس فكرة

One-to-Many

مثال:

Category → Products

```
class Category
{
```

```
    public List<Product> Products;
}

class Product
{
    public int CategoryId;
}
```

🔗:المعنى

- واحد Category
- عنده منتجات كثيرة

🔗 One-to-One

مثال:

User → Profile

```
class User
{
    public UserProfile Profile;
}
```

🔗:المعنى

واحد مقابل واحد

Many-to-Many

مثال:

Student ↔ Course

- الطالب يأخذ عدة كورسات
- الكورس فيه عدة طلاب

تلقائي EF Core يسوي Join Table

مهم:

ICollection<Course> Courses

Many هذا يعني علاقة

Join Table (Advanced)

```
class WorksOn
{
    public int Essn;
    public int Pno;
}
```

هذا يربط بين جدولين

7) InverseProperty (حل مشكلة تعارض العلاقات)

💡 الفكرة:

إذا عندك أكثر من علاقة بين نفس الكلاسين

مثال:

- Manager
- Employees

يحتاج يفهم كل علاقة لحالها EF

8) Self Relationship

💡 كلاس يرتبط بنفسه

مثال:

Employee → Supervisor

```
public Employee Supervisor;  
public List<Employee> Supervisees;
```

💡 الفكرة:

موظف ممكن يكون مدير وموظف بنفس الوقت

(مقابلة مهم جداً) EF Core أهم فكرة في 🧠

💡 الجملة الذهبية:

EF Core maps C# classes to database tables and handles relationships automatically using conventions and annotations.

🔍 الفرق الحقيقي الذي يجب تفهمه

بدون EF Core:

- SQL تكتب
- تدير الجداول يدوي

مع EF Core:

- فقط Objects تتعامل مع
- EF Core يترجم للباقي

(ملخص سريع جداً) (احفظه) 🧠

EF Core = 3 أشياء:

1. DbContext → مدير الداتابيس
2. DbSet → Table
3. Entity → Class

العلاقات:

العلاقة	المعنى
One-to-One	واحد ↔ واحد
One-to-Many	واحد ↔ كثير
Many-to-Many	كثير ↔ كثير